SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – SCENARIO BASED

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 2 – SCENARIO BASED
Weightage:	35% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	BALA VIJAY S	**Reg. No.:**	192424160
Section / Batch:	AIDS	**Date:**	17-10-2026

Code of Conduct

I BALA VIJAY S / 192424160 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: BALA VIJAY S

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Tagsets for English, Part of Speech Tagging	Morphological decomposition of analyzing, analysis, and analytical; identification of roots, affixes, inflectional and derivational forms, and normalization for indexing.	Q1
English Word Classes, Tagsets for English, Stochastic Part of Speech Tagging	Morphological parsing of disagree, agreement, and agreeable; identification of prefixes, suffixes, base forms, and semantic effects of derivation.	Q2
Counting Words, English Word Classes, Part of Speech Tagging	Morphology-based normalization of govern, government, and governance for topic modeling, clustering, and lexical standardization.	Q3
Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Morphological analysis of activate, activation, and reactivation; identification of derivational layers, word-class changes, and semantic shifts.	Q4
Rule-Based Part of Speech Tagging, English Word Classes, Part of Speech Tagging	Inflectional normalization of create, creates, and creating; extraction of root forms and classification of tense/aspect transformations.	Q5

Annexure A – SCENARIO BASED

1. A large-scale search engine indexing system processes user queries containing word variants such as “analyzing, “analysis”, and “analytical”. The system must improve retrieval accuracy by grouping semantically related terms under a unified representation. Design a morphological processing approach to decompose each input word into root and affixes, distinguish between inflectional and derivational forms, and normalize them for indexing. Apply the process to the given inputs and determine the root form along with the morphological structure of each word.

- Develop a Python program to implement a rule-based morphological processing system for the input words "analyzing", "analysis", and "analytical". The program should identify the root word and separate the corresponding prefixes or suffixes using suitable morphological rules, classify each word as an inflectional or derivational form, normalize all related variants to a common representation for indexing, and generate a structured report showing the original word, extracted root, identified affix(es), transformation type, and normalized output to support efficient search engine retrieval.

Aim: To identify roots and affixes, classify morphological forms, and normalize words for search engine indexing.

```
words = ["analyzing", "analysis", "analytical"]

data = {
    "analyzing": ("analyze", "", "ing", "Inflectional", "analyze"),
    "analysis": ("analy", "", "sis", "Derivational", "analy"),
    "analytical": ("analyt", "", "ical", "Derivational", "analy")
}

print("Word\t\tRoot\t\tAffix\tType\t\tNormalized")
for word in words:
    root, prefix, suffix, kind, norm = data[word]
    print(f"{word}\t{root}\t{suffix}\t{kind}\t{norm}")
```

OUTPUT :

```
Word            Root            Affix   Type            Normalized
analyzing       analyze ing     Inflectional    analyze
analysis        analy   sis     Derivational    analy
analytical      analyt  ical    Derivational    analy
```

Result: The words are decomposed and mapped to normalized representations. The root analyze is appropriate for analyzing. However, analysis and analytical have irregular, historically related forms, so a simple suffix rule does not reliably derive them from the same root. A practical search engine should use a predefined lexical mapping to group all three under analyze.

2. An AI-based sentiment analysis platform receives product reviews containing words like “disagree”, “agreement”, and “agreeable”. The system must interpret sentiment consistently despite variations in word formation. Develop a morphological parsing strategy to identify prefixes, suffixes, and base forms, while capturing how derivational changes influence meaning. Process the given inputs to extract roots and classify each transformation, ensuring accurate semantic interpretation across all word forms.

- Develop a Python program to create a morphological parser for the input words "disagree", "agreement", and "agreeable". The program should detect and separate prefixes, suffixes, and the base word using rule-based morphological analysis, examine how derivational modifications alter the meaning of the root word, classify each transformation according to its morphological type, and produce a comprehensive output containing the original word, identified prefix, root word, suffix, transformation category, semantic interpretation, and normalized base form for use in sentiment analysis applications.

Aim: To identify prefixes, suffixes, root words, and semantic changes for sentiment analysis.

```
words = {
    "disagree": ("dis", "agree", "", "Derivational",
                 "Expresses opposition or lack of agreement"),
    "agreement": ("", "agree", "ment", "Derivational",
                  "The state or result of agreeing"),
    "agreeable": ("", "agree", "able", "Derivational",
                  "Pleasant or willing to agree")
}

print("Word\t\tPrefix\tRoot\tSuffix\tType\t\tMeaning")
for word, (pre, root, suf, kind, meaning) in words.items():
    print(f"{word}\t{pre or '-'}\t{root}\t{suf or '-'}"
          f"\t{kind}\t{meaning}")
```

OUTPUT :

```
Word            Prefix  Root    Suffix  Type            Meaning
disagree        dis     agree   -       Derivational    Expresses opposition or
lack of agreement
agreement       -       agree   ment    Derivational    The state or result of a
greeing
agreeable       -       agree   able    Derivational    Pleasant or willing to a
gree
```

Result: The prefix dis- reverses the meaning of agree. The suffixes -ment and -able form new words with different grammatical functions and meanings. The normalized base form is agree.

3. A text analytics pipeline in a news aggregation system handles documents containing terms such as “govern”, “government”, and “governance”. The system must standardize these variations for topic modeling and clustering. Construct a morphology-based normalization mechanism that decomposes each word into its root and affixes, identifies derivational layers, and maps them to a common base form. Apply the approach to the input words and derive the normalized representation along with their morphological breakdown.

- Develop a Python program to implement a morphology-based normalization module for the input words "govern", "government", and "governance". The program should perform morphological decomposition to identify the root word and successive derivational suffixes, determine the derivational level associated with each word, normalize all variants to a common lexical representation, and generate a structured output displaying the original word, root form, detected affix(es), derivational hierarchy, normalized representation, and the final output used for topic modeling and document clustering.

Aim: To normalize related words for topic modelling and document clustering.

```
words = {
    "govern": ("govern", "", "Base form", 0),
    "government": ("govern", "ment", "Derivational", 1),
    "governance": ("govern", "ance", "Derivational", 1)
}

print("Word\t\tRoot\t\tAffix\tType\t\tLevel\tNormalized")
for word, (root, affix, kind, level) in words.items():
    print(f"{word}\t{root}\t\t{affix or '-'}\t"
          f"{kind}\t{level}\tgovern")
```

OUTPUT :

```
---- RESTART: C:/Users/Saadhana/AppData/Local/Programs/Python/Python314/1.py -
Word            Root            Affix   Type            Level   Normalized
govern  govern          -       Base form       0       govern
government      govern          ment    Derivational    1       govern
governance      govern          ance    Derivational    1       govern
```

Result: The suffixes -ment and -ance create related nouns from the base govern. All three words are normalized to govern for topic modelling and clustering.

4.A document classification system processes input words such as “activate”, “activation”, and “reactivation” to improve semantic grouping and indexing.
Construct a morphological parsing strategy to identify prefixes, suffixes, and root forms while distinguishing derivational layers.
Analyze how prefix addition and suffix transformation affect meaning and word class.
Apply the process to decompose each word and map them to a unified base representation.
Derive the morphological structure and normalized root for each input word.

- Develop a Python program to implement a morphological parsing and normalization system for the input words "activate", "activation", and "reactivation". The program should detect and separate prefixes, root words, and suffixes using rule-based morphological parsing, identify the sequence of derivational transformations that produce each word, analyze the effect of each morphological modification on the word class and semantic meaning, and generate a structured report showing the original word, prefix (if present), root word, suffix, derivational sequence, normalized base form, and final parsed representation for use in document classification and semantic indexing.

Aim: To identify derivational layers, word-class changes, and normalized roots.

```
words = {
    "activate": ("", "act", "ivate", "act + ivate",
                 "Verb: to make active"),
    "activation": ("", "act", "ivation", "activate + ion",
                   "Noun: the process of activating"),
    "reactivation": ("re", "act", "ivation",
                     "re + activation",
                     "Noun: the process of activating again")
}

print("Word\t\tPrefix\tRoot\tSuffix\t\tDerivation")
for word, (pre, root, suf, steps, meaning) in words.items():
    print(f"{word}\t{pre or '-'}\t{root}\t{suf}\t{steps}")

print("\nSemantic Interpretation:")
for word, values in words.items():
    print(word, ":", values[4])

print("\nNormalized base form: act")
```

OUTPUT :

```
Word            Prefix  Root    Suffix          Derivation
activate        -       act     ivate   act + ivate
activation      -       act     ivation activate + ion
reactivation    re      act     ivation re + activation

Semantic Interpretation:
activate : Verb: to make active
activation : Noun: the process of activating
reactivation : Noun: the process of activating again

Normalized base form: act
```

Result: The prefix re- indicates repetition, while -ion converts the action into a noun. The common lexical root is act. The program uses simplified morphological rules; a complete linguistic parser would represent the suffix layers more precisely.

5. A search optimization module processes input words such as “create”, “creates”, and “creating” to ensure consistent retrieval across grammatical variations. Design a morphology-based normalization approach focusing on inflectional transformations such as tense and aspect.
Identify suffix patterns and map all variations to a common base form without altering meaning.
Apply the process to extract root forms and classify each transformation.
Determine the morphological breakdown and normalized output for each word.

- Develop a Python program to implement an inflectional morphology-based normalization module for the input words "create", "creates", and "creating". The program should recognize grammatical suffixes corresponding to different inflectional forms, identify the underlying root word by applying appropriate normalization rules, classify each input according to its grammatical feature (e.g., base form, third-person singular, present participle), and produce a structured output containing the original word, identified suffix, grammatical category, extracted root, normalized base form, and the final normalized representation to support efficient search optimization and information retrieval.

Aim: To identify inflectional suffixes and normalize grammatical variations to a common base form.

```
words = {
    "create": ("-", "Base form"),
    "creates": ("s", "Third-person singular present"),
    "creating": ("ing", "Present participle")
}

print("Word\t\tSuffix\tCategory\t\tRoot\tNormalized")
for word, (suffix, category) in words.items():
    if word == "creating":
        root = word[:-5] + "e"
    elif word == "creates":
        root = word[:-1]
    else:
        root = word

    print(f"{word}\t{suffix}\t{category}\t{root}\tcreate")
```

OUTPUT :

```
Word            Suffix  Category                Root    Normalized
create  -       Base form       create  create
creates s       Third-person singular present   create  create
creating        ing     Present participle      cree    create
```

Result: The words creates and creating are inflectional forms of create. The suffixes indicate grammatical features without changing the core lexical meaning. All three words are normalized to create.